

Short Answer Questions

1. primitive; reference
2. superclass; subclass
3. reference; content / logical
4. abstract class; interface
5. ArrayList; LinkedList
6. Comparable
7. reference; Integer
8. <E> void print(E[] list)
9. raw type
10. sequential / linear; index
11. Comparable; Comparator
12. poll(); remove()
13. [1, 3, 5, 2, 4, 6]
14. Charlie
15. 2
16. smallest; first unsorted
17. linear; logarithmic
18. dynamic programming; divide-and-conquer
19. $O(n^2)$
20. [9, 4, 12, 15]
21. The value depends on the visible heap figure. Choose a value smaller than its parent and larger than its children to preserve max-heap order.
22. complete; incomplete
23. DFS; BFS
24. Prim's algorithm; Dijkstra's algorithm
25. preorder; inorder
26. Depends on the tree figure in the original image. Postorder = left, right, root. Breadth-first = level order.
27. RR; LL
28. Rotation depends on the visible AVL figure after inserting 81.
29. index / hash value; separate chaining
30. double; clustering

Coding Question 1 - LibraryManagementSystem

```
import java.util.Arrays;
import java.util.HashSet;
import java.util.PriorityQueue;
import java.util.Set;

public class LibraryManagementSystem {
    public static void main(String[] args) {
        PriorityQueue<String> fictionQueue = new PriorityQueue<>(
            Arrays.asList("The Great Gatsby", "To Kill a Mockingbird", "1984",
                "Pride and Prejudice", "Harry Potter", "The Hobbit"));
        PriorityQueue<String> historicalQueue = new PriorityQueue<>(
            Arrays.asList("Sapiens", "Pride and Prejudice",
                "A Brief History of Time", "Educated"));
        PriorityQueue<String> referenceQueue = new PriorityQueue<>(
            Arrays.asList("The Great Gatsby", "A Brief History of Time",
                "Harry Potter", "Encyclopedia Britannica", "The Hobbit"));

        System.out.println(findPopularBooks(fictionQueue, historicalQueue, referenceQueue));
    }
}
```

```

    public static Set<String> findPopularBooks(
        PriorityQueue<String> fictionQueue,
        PriorityQueue<String> historicalQueue,
        PriorityQueue<String> referenceQueue) {
        Set<String> popularBooks = new HashSet<>(fictionQueue);
        popularBooks.retainAll(historicalQueue);
        popularBooks.retainAll(referenceQueue);
        return popularBooks;
    }
}

```

For the given data, no book appears in all three queues, so the result is [] / empty set.

Coding Question 2 - EmployeeRecordSorting

```

import java.util.Arrays;
import java.util.Comparator;

public class EmployeeRecordSorting {
    public static <E extends Comparable<E>> void insertionSort(E[] list) {
        for (int i = 1; i < list.length; i++) {
            E current = list[i];
            int k;
            for (k = i - 1; k >= 0 && list[k].compareTo(current) > 0; k--) {
                list[k + 1] = list[k];
            }
            list[k + 1] = current;
        }
    }

    public static <E> void insertionSort(E[] list, Comparator<? super E> comparator) {
        for (int i = 1; i < list.length; i++) {
            E current = list[i];
            int k;
            for (k = i - 1; k >= 0 && comparator.compare(list[k], current) > 0; k--) {
                list[k + 1] = list[k];
            }
            list[k + 1] = current;
        }
    }

    public static void main(String[] args) {
        Integer[] employeeIds = {102, 101, 104, 103, 105};
        String[] employeeNames = {"John", "Alice", "bob", "Diana", "Charlie"};

        insertionSort(employeeIds);
        System.out.println(Arrays.toString(employeeIds));

        insertionSort(employeeNames, String.CASE_INSENSITIVE_ORDER);
        System.out.println(Arrays.toString(employeeNames));
    }
}

```

Expected sorted IDs: [101, 102, 103, 104, 105]

Expected sorted names: [Alice, bob, Charlie, Diana, John]